Chapter 08: Spring AOP

Department of Software Engineering
Faculty of Information Technology (HCM-UTE)
Course: Web Programming
Course ID: WEPR330479
Lecturer: Msc. Khoa Hoang

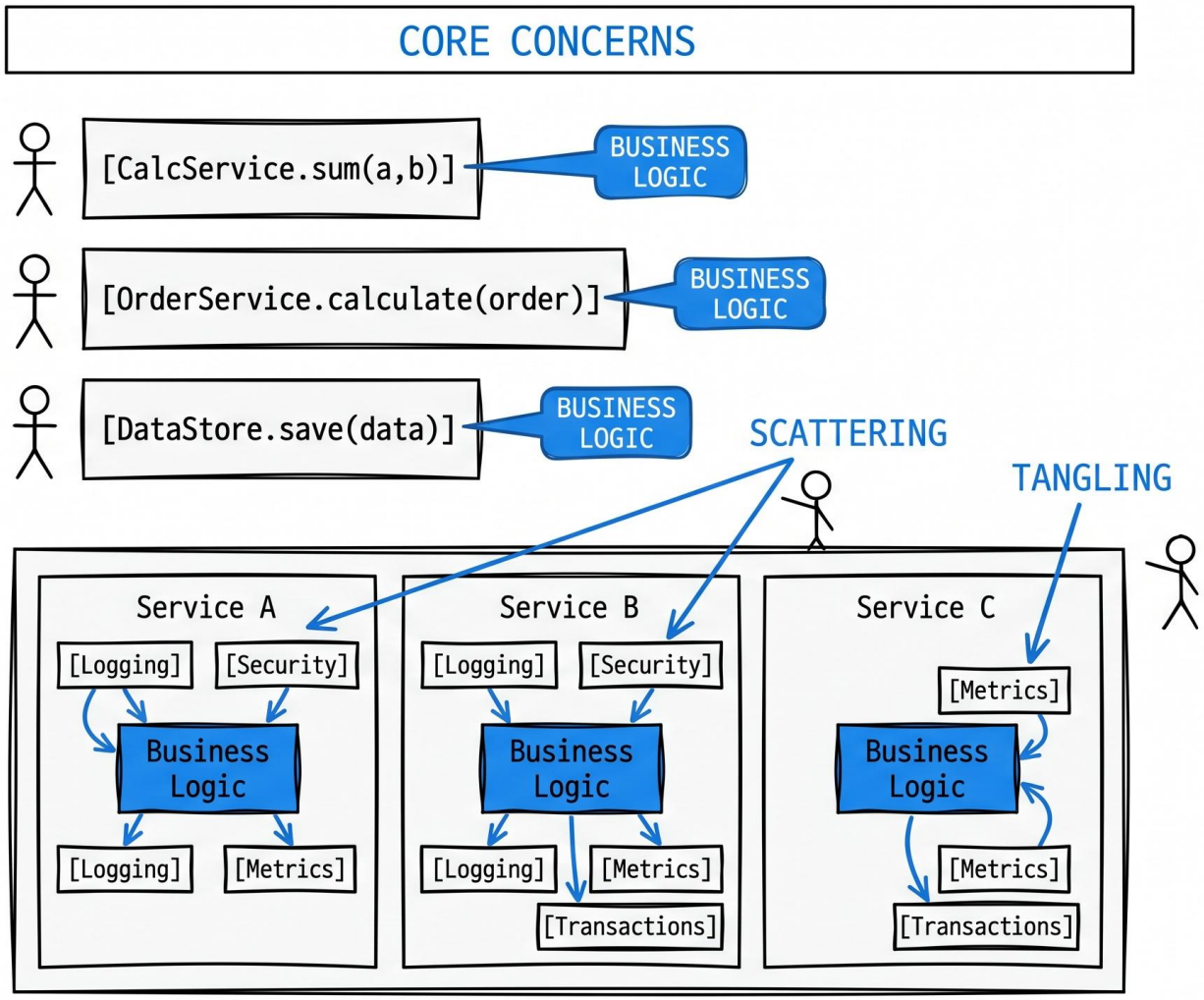
Content

- Introduction to AOP
- Core AOP concepts
- Advice types
- Pointcut expressions

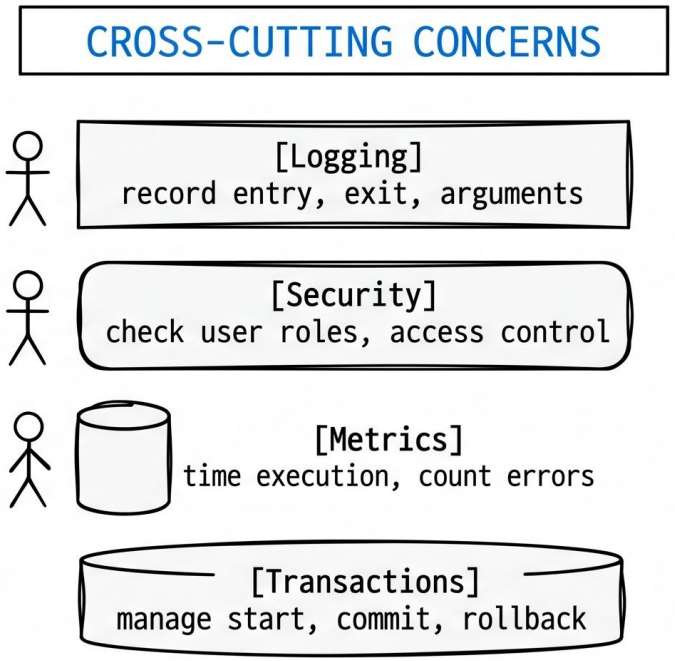
Introduction to Aspect-Oriented Programming (AOP)

Modularizing cross-cutting concerns across your application

Concerns



Scattering & Tangling Diagram



- Spring AOP separates core from cross-cutting concerns.
- Aspects capture modules of concerns across multiple objects.
- Advice defines *when* and *how* to apply the concern.
- Joinpoints are the spots in execution where advice can be inserted.
- Pointcuts identify specific joinpoints to match advice.

Example #1

```
package com.example.demo;

public class HelloWorld {

    public void sayHello() {}

}
```



```
package com.example.demo;

public class HelloWorld {

    public void sayHello() {
        System.out.println("before sayHello");

        // business logic code
        // ...

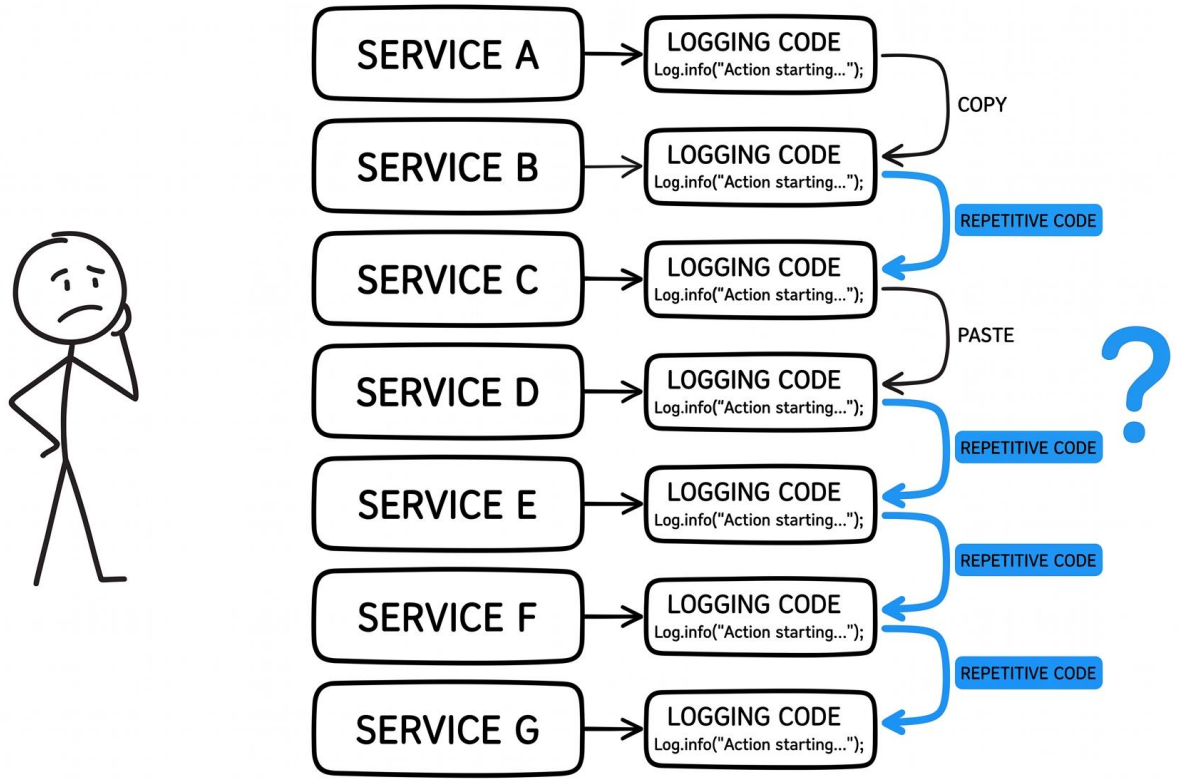
        System.out.println("after sayHello");
    }

}
```

Need to copy/paste

Mixing business logic code with other code types

Pain point



IF 50 SERVICES NEED LOGGING, WILL YOU COPY-PASTE 50 TIMES?

Example #2

```
package com.example.demo;

public class HelloWorld {

    public void sayHello() {}

}
```



```
package com.example.demo;

public class HelloWorldProxy extends HelloWorld {

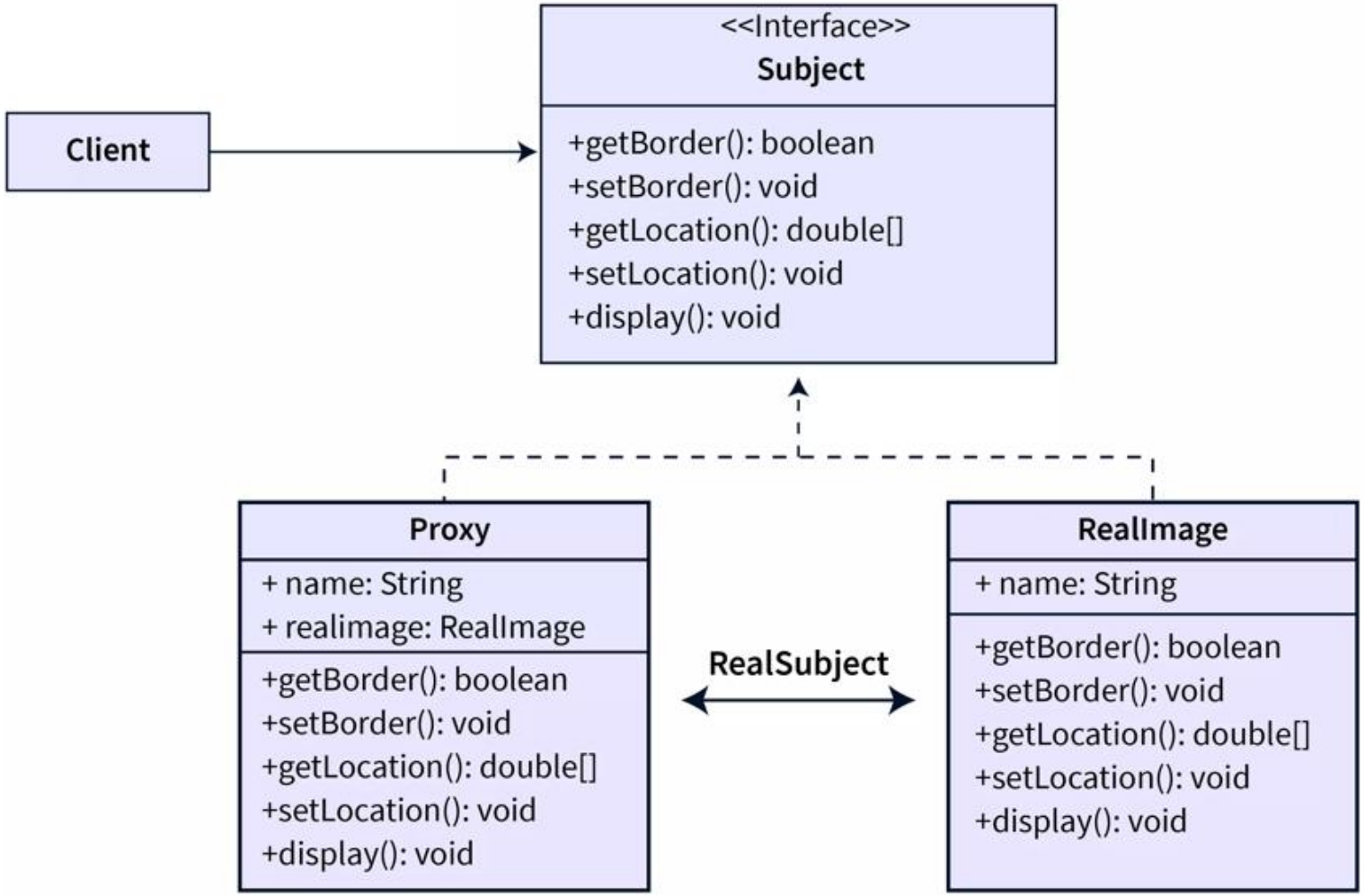
    private void beforeSayHello() {
        System.out.println("before call the sayHello() method");
    }

    @Override
    public void sayHello() {
        beforeSayHello();
        super.sayHello();
        afterSayHello();
    }

    private void afterSayHello() {
        System.out.println("after called the sayHello() method");
    }
}
```

The basic idea is to create a class (acts as a Proxy class) that wraps an existing class in order to add actions before and after function calls:
HelloWorld -> HelloWorldProxy

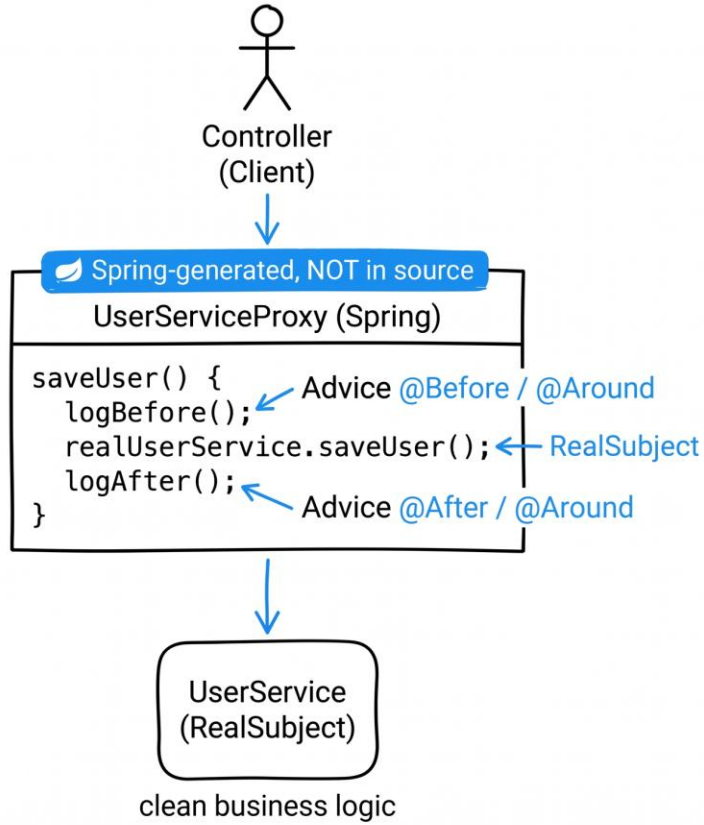
The Proxy pattern



Example

```
@Service
public class UserService {
    public void saveUser() { ... }
}
```

```
@Aspect
@Component
public class LoggingAspect {
    @Around("execution(* com.example.demo.service.*.*(..))")
    public Object log(ProceedingJoinPoint pjp) throws Throwable {
        logBefore();
        Object result = pjp.proceed();
        logAfter();
        return result;
    }
}
```



What is Aspect-Oriented Programming?

- AOP complements Object-Oriented Programming (OOP)
- OOP organizes systems into hierarchies of classes
- AOP encapsulates cross-cutting concerns into Aspect classes
- Separates boilerplate infrastructure from business rules
- Spring AOP extracts repetitive concerns like logging and transactions into reusable aspects, keeping your business services clean, modular, and focused.
- Official documentation: https://docs.spring.io/spring-framework/reference/core/aop.html

JDK Dynamic Proxies vs CGLIB

- JDK Dynamic Proxy: used when target class implements an interface
- CGLIB (Code Generation Library): generates subclass bytecode at runtime for classes
- Spring Boot enables CGLIB by default (proxyTargetClass = true), works seamlessly on both interfaces and concrete classes

Real-world Spring AOP examples

- @Transactional: opens, commits or rolls back DB transactions
- @PreAuthorize: checks user role before invoking service method
- @Cacheable: returns cached result or executes and stores value
- Custom audit logs: recording user ID, timestamp and changes

Benefits of AOP

- Cleaner code: business methods contain only business logic
- DRY principle: write logging or timing logic exactly once
- Centralized maintenance: change policy in a single aspect
- Plug-and-play: enable or disable features declaratively

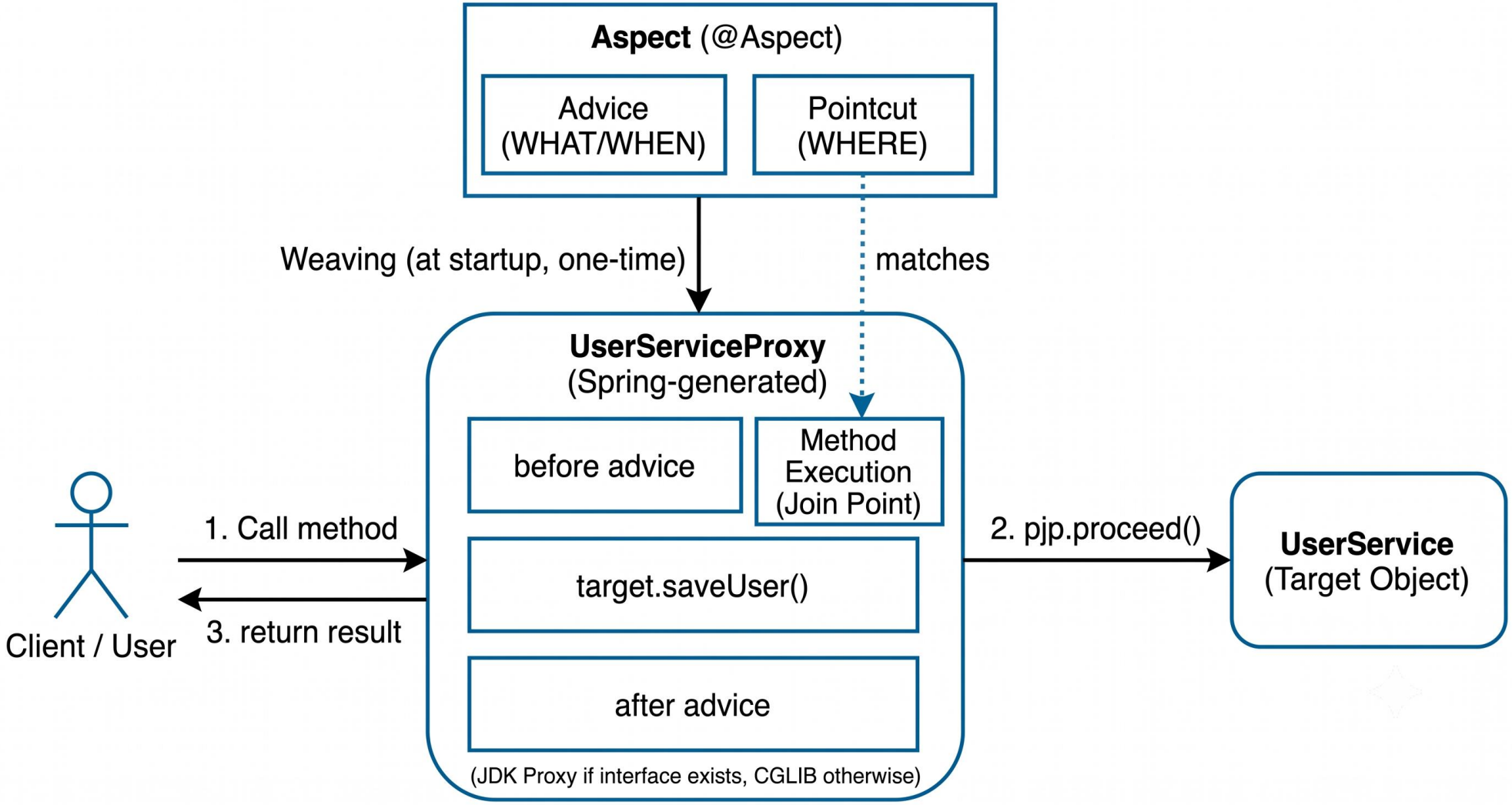
Core AOP concepts

Key terminology and relationships in Spring AOP

Core concepts #1

Concept	Description
Aspect	A module encapsulating a cross-cutting concern A class with the @Aspect annotation, indicating a cross-cutting concern.
Join point	A specific point during program execution In Spring AOP, a Join Point is always a method execution
Pointcut	An expression matching one or more Join Points Defines WHERE the cross-cutting action should occur
Advice	The actual action taken at a matched Join Point Defines WHAT should run and WHEN (before, after, around)
Target object	The bean being advised by one or more aspects
AOP proxy	The generated wrapper object executing the aspect
Weaving	It is the process of linking aspects with target objects to create proxies Spring AOP performs runtime weaving via dynamic proxies

Core concepts #2



Advice types

#	Annotation	Runs when	Can block method?	Can modify result?
1	@Before	Before the method	No	No
2	@After	After the method (even on exception)	No	No
3	@AfterReturning	After the method returns successfully	No	No
4	@AfterThrowing	After the method throws an exception	No	No
5	@Around	Around the method (wraps the method)	Yes	Yes

@Before annotation

```
@Aspect
@Component
public class LoggingAspect {
    @Before("execution(* com.example.demo.service.*.*(..))")
    public void logBefore(JoinPoint jp) {
        System.out.println("[BEFORE] Call method: " + jp.getSignature().getName());
    }
}
```

Runs BEFORE the original method executes.

Real-world use cases:

- Log the start of the operation.
- Check permissions (if invalid -> throw an exception to halt execution).
- Validate input.

Characteristics:

- Includes a JoinPoint parameter to access method information.
- Cannot modify the result.
- Cannot block the method (unless an exception is thrown).

@After annotation

```
@Aspect
@Component
public class LoggingAspect {
    @After("execution(* com.example.demo.service.*.*(..))")
     public void logAfter(JoinPoint jp) {
        System.out.println("[AFTER] Call method: " + jp.getSignature().getName());
     }
}
```

Runs AFTER the original method executes.

Real-world use cases:

- Release resources: close files, connections…
- Log completion.

Characteristics:

- **@AfterReturning** executes only when the method returns successfully (without throwing an exception).
- **@AfterThrowing** executes only when the method throws an exception.

@Around annotation

```
@Aspect
@Component
public class LoggingAspect {
    @Around("execution(* com.example.demo.service.*.*(..))")
    public Object logAround(ProceedingJoinPoint pjp) throws Throwable {
        // BEFORE
        System.out.println("[BEFORE] " + pjp.getSignature());

        Object result;
        try {
            result = pjp.proceed(); // <- call the original method
        } catch (Throwable ex) {
            // AFTER THROWING
            System.out.println("[ERROR] " + ex.getMessage());
            throw ex; // <- very important, must re-throw
        } finally {
            // AFTER
            System.out.println("[AFTER] " + pjp.getSignature());
        }

        return result;
    }
}
```

Notes:

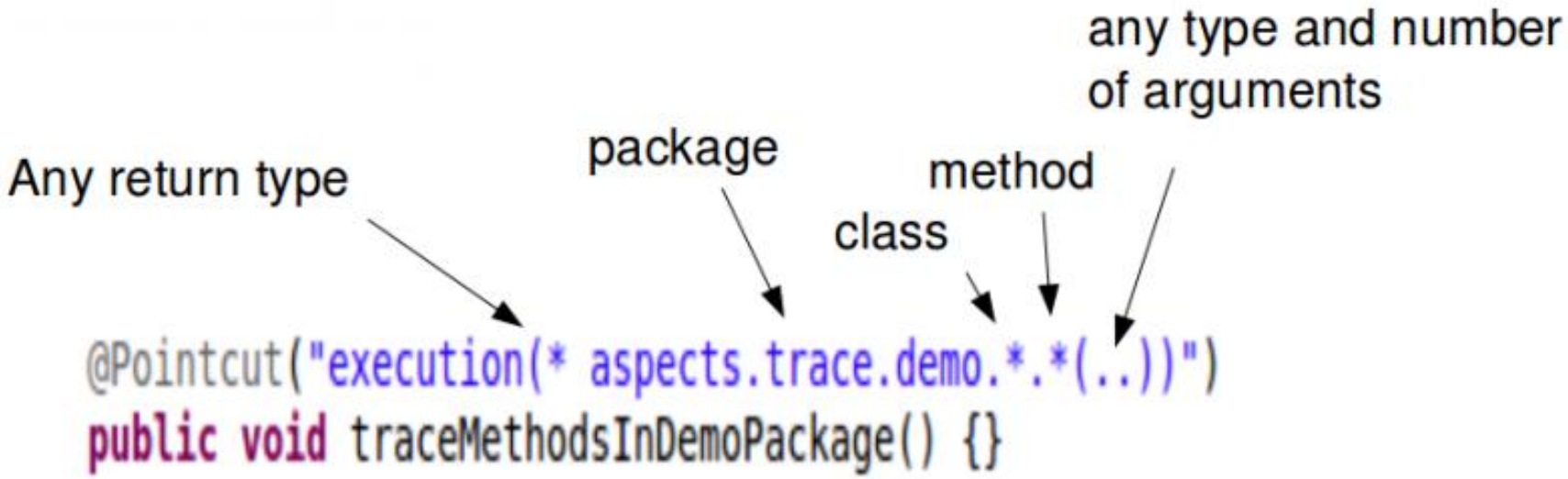
- The parameter is `ProceedingJoinPoint` (not `JoinPoint`).
- You must call `pjp.proceed()` to execute the original method.
- `proceed()` returns an `Object`, so you need to return it.
- If you don't call `proceed()`, the original method will NOT execute.
- You can modify the result before returning it.

Comparison table: when to use

#	Annotation	When to use?
1	@Before	Need to run before the method, don't care about the result
2	@After	Always need to run at the end (whether success or failure)
3	@AfterReturning	Need to process the returned result
4	@AfterThrowing	Need to handle when an error occurs
5	@Around	Need full control (block, modify result)

Pointcut expressions

The execution() designator



Pointcut wildcards

- * matches any return type or single package/class name
- .. in package path matches current package and all sub-packages
- .. in parameter list matches zero, one, or multiple arguments
- (*) matches exactly one argument of any type

Common pointcut patterns

- execution(* com.example.demo.service.*.*(..)): all service methods
- execution(* com.example..*Repository.*(..)): all repositories
- Combine pointcuts using logical operators: &&, ||, and !
  Example: @Around("execution(* com.example.service.*.*(..)) && !@annotation(NoLog)")
  Log all methods in the service, except those annotated with @NoLog.
- @Pointcut: declare reusable named pointcut methods

Q&A – Thank you